

```
#include <stdio.h>
```

```
#include <string.h>
```

```
#include <ctype.h>
```

```
#define MAX 100
```

```
char stack[MAX];
```

```
int top = -1;
```

```
void push(char item) {
```

```
    if (top == (MAX - 1))
```

```
        printf("Stack Overflow\n");
```

```
    else
```

```
        stack[++top] = item;
```

```
}
```

```
char pop() {
```

```
    if (top == -1) {
```

```
        printf("Stack Underflow\n");
```

```
        return '\0';
```

```
    }
```

```
    return stack[top--];
```

```
}
```

```
int precedence(char symbol) {  
    switch (symbol) {  
        case '^': return 3;  
        case '*': case '/': return 2;  
        case '+': case '-': return 1;  
        default: return 0;  
    }  
}
```

```
void infixToPostfix(char infix[], char postfix[]) {  
    int i = 0, j = 0;  
    char symbol;  
    push('(');  
    strcat(infix, " ");  
  
    while (infix[i] != '\0') {  
        symbol = infix[i];  
  
        if (symbol == ' ') {  
            i++;  
            continue;  
        }  
  
        if (symbol == '(')  
            push(symbol);
```

```

else if (isalnum(symbol))
    postfix[j++] = symbol;
else if (symbol == ')') {
    while (top != -1 && stack[top] != '(')
        postfix[j++] = pop();
    pop();
}
else {
    while (top != -1 && precedence(stack[top]) >= precedence(symbol))
        postfix[j++] = pop();
    push(symbol);
}
i++;
}
postfix[j] = '\0';
}

```

```

int main() {
    char infix[MAX] = "A + B * C";
    char postfix[MAX];

    printf("Infix Expression: %s\n", infix);

    infixToPostfix(infix, postfix);

    printf("Postfix Expression: %s\n", postfix);
}

```

```
return 0;
```

```
}
```